

Wykonaj aplikację konsolową oraz desktopową według wskazań. Wykonaj dokumentację do utworzonych aplikacji zgodnie z opisem

Algorytm wyszukiwania binarnego – krótkie wyjaśnienie

Algorytm wyszukiwania binarnego służy do szybkiego znajdowania elementu w posortowanej tablicy lub liście.

Zasada działania:

1. Sprawdzamy element znajdujący się w środku zbioru.
2. Porównujemy go z szukaną wartością.
3. Jeśli wartości są równe – element został znaleziony.
4. Jeśli szukana wartość jest mniejsza – przeszukujemy lewą połowę.
5. Jeśli większa – przeszukujemy prawą połowę.
6. Czynność powtarzamy aż do znalezienia elementu lub wyczerpania zakresu.

Przykład:

Tablica: [2, 5, 8, 12, 16, 23, 38]

Szukana liczba: 16

1. Środek = 12 → szukamy w prawej połowie.
2. Środek = 23 → szukamy w lewej połowie.
3. Środek = 16 → znaleziono.

Część I. Aplikacja konsolowa

- Napisz program w C++, który:
 1. Wczyta z pliku *dane.txt* ciąg liczb całkowitych (każda liczba jest zapisana w nowej linii)
 2. Poprosi użytkownika o podanie liczby do wyszukania (klawiatura).
 3. Posortuje wczytane dane własnym algorytmem sortowania (bez funkcji bibliotecznych do sortowania).
 4. Wykona wyszukiwanie binarne i poda:
 - indeks znalezionego elementu (w posortowanej tablicy) albo informację, że element nie występuje.
- Wymagania organizacyjne:
 - Każdy etap ma być zrealizowany w oddzielnej funkcji/metodzie:
 - wczytanie danych,
 - sortowanie,
 - wyszukiwanie binarne,

Podaj liczbę do wyszukania:5

Posortowane dane: 1 2 3 4 5 6 7 7 8 9 10 11 12 13 14

Znaleziono 5 na indeksie 4 (w posortowanej tablicy).

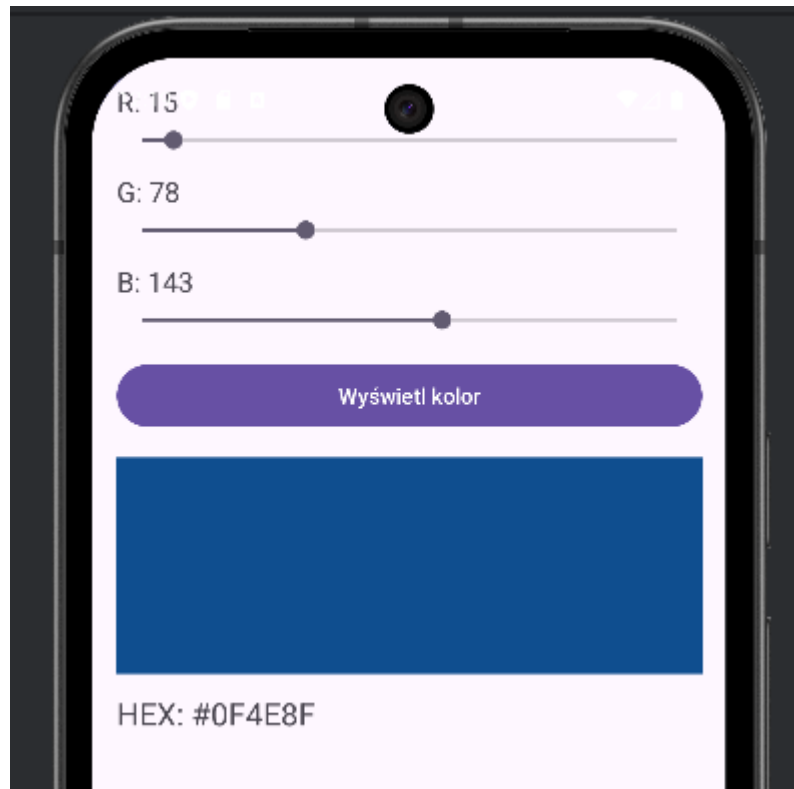
- pobranie liczby od użytkownika,
- wypisanie wyniku.

Część II. Aplikacja mobilna

Aplikacja mobilna „RGB Color Picker”

Wykonaj aplikację Android mobilną, która pozwala użytkownikowi wybrać kolor za pomocą trzech suwaków RGB i wyświetlić wynik.

- Aplikacja ma umożliwiać ustawienie wartości kanałów:
 - R (Red – czerwony)
 - G (Green – zielony)
 - B (Blue – niebieski)
- Każdy kanał ma przyjmować wartości z zakresu 0–255. Po ustawieniu suwaków i naciśnięciu przycisku aplikacja ma:
 1. Wyświetlić wybrany kolor w wyznaczonym elemencie typu View (np. prostokąt/pole koloru).
 2. Wyświetlić zapis szesnastkowy wybranego koloru w polu tekstowym, w formacie np. #RRGGBB.
- **Wymagania funkcjonalne**
 1. W aplikacji znajdują się trzy suwaki (SeekBar):
 - suwak R (0–255)
 - suwak G (0–255)
 - suwak B (0–255)
 2. Przy suwakach powinny być widoczne aktualne wartości liczbowe (np. w TextView obok każdego suwaka), aby użytkownik widział ustawione liczby.
 3. W aplikacji znajduje się przycisk: „Wyświetl kolor”.
 4. Po kliknięciu przycisku:
 - aplikacja oblicza kolor z aktualnych wartości R, G, B,
 - ustawia tło wskazanego elementu View na ten kolor,
 - wyświetla w polu tekstowym wartość koloru w systemie szesnastkowym (HEX), np. #1A7F00.



Część III. Dokumentacja kodu aplikacji konsolowej

- Przygotuj **dokumentację do kodu** (mogą być komentarze w stylu Doxygen), tak aby osoba czytająca program rozumiała:
 - kto jest autorem,
 - do czego służy każda funkcja,
 - jakie funkcja przyjmuje argumenty,
 - co zwraca (lub że zwraca void),

Co musi zawierać dokumentacja

1. **Nagłówek pliku na początku:**
 - autor (imię i nazwisko),
 - data,
 - krótki opis programu (co robi całość).
2. **Dokumentacja każdej funkcji (obowiązkowo):**
 - krótki opis zadania funkcji,
 - @param dla każdego argumentu (co oznacza, jakie ograniczenia),
 - @return – co zwraca; jeśli nic nie zwraca, jasno: void,

Przykład dokumentacji (fragment):

```
/**
 * @file stats_example.cpp
 * @author Jan Kowalski
 * @date 2026-02-15
 * @brief Przykład dokumentowania funkcji w C++ (format Doxygen).
 *
 * Plik zawiera kilka prostych funkcji do obliczeń na liczbach
 * oraz przykład funkcji niezwracającej wartości (void).
 */

#include <iostream>
#include <vector>
#include <stdexcept>

/**
 * @brief Oblicza średnią arytmetyczną liczb z wektora.
 *
 * Funkcja sumuje elementy wektora i dzieli przez ich liczbę.
 * Jeśli wektor jest pusty, zgłasza wyjątek.
 *
 * @param values Wektor liczb typu double. Nie może być pusty.
 * @return Średnia arytmetyczna elementów wektora.
 * @throws std::invalid_argument Gdy values jest pusty.
 */

double average(const std::vector<double>& values) {
    if (values.empty()) {
        throw std::invalid_argument("values cannot be empty");
    }

    double sum = 0.0;
    for (double v : values) sum += v;

    return sum / static_cast<double>(values.size());
}

//[...]
```